МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 6-05-0611-01 «Информационные системы и технологии»

Дисциплина Операционные системы

Лабораторная работа № 3
УПРАВЛЕНИЕ ПРОЦЕССАМИ

Студент 3 курса 2 группы
Марушко Т. Ф.
Проверил: Ассистент Савельева М. Г.

Минск, 2026

Содержание

Введение....3
1 Выполнение заданий на ОС *Linux*....4
1.1 Подготовка среды....4
1.2 Уровень 4–5: базовые операции с процессами....4
1.3 Уровень 7–8: *IPC* через анонимный канал....8
1.4 Уровень 9–10: обработка сигналов *SIGINT*....10
1.5 Вывод по разделу....11
2 Выполнение заданий на ОС *Windows*....12
2.1 Подготовка среды и сборка....12
2.2 Уровень 4–5: создание процессов (*CreateProcess*)....12
2.3 Уровень 7–8: *IPC* через канал и *STARTUPINFO*....16
2.4 Уровень 9–10: передача через *STARTUPINFO*....17
2.5 Вывод по разделу....19
Заключение....20

Введение

Цель лабораторной работы – получение практических навыков в управлении процессами в операционных системах *Windows* и *Linux*, включая создание и контроль процессов, межпроцессное взаимодействие (*IPC*) и передачу данных через структуру *STARTUPINFO*.

Для достижения цели определены задачи:

- написать программу *Lab*-03*x* (цикл с задержкой 500 мс, вывод *PID*, аргумент командной строки или переменная окружения *ITER_NUM*);
- реализовать *Lab*-03*a*: создание трёх дочерних процессов *CreateProcess* с разными способами передачи параметров;
- реализовать *Lab*-03*b*: вариант с глобальной и локальной переменной окружения *ITER_NUM*;
- написать *Lab*-03*c*: вывод списка процессов через *Tool Help Library* (*PID*, *PPID*, имя);
- реализовать пару *Lab*-03*d-client* / *Lab*-03*d-server*: поиск чисел, взаимно простых с *M*, через анонимный канал и *STARTUPINFO*;
- реализовать *Lab*-03*hello-parent* / *Lab*-03*hello-child*: передача строки через *STARTUPINFO* (*lpTitle* + *CREATE_NEW_CONSOLE*);
- написать аналогичные программы для *Linux* (*Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*sig* — устойчивость к *SIGINT*);
- проверить работу всех приложений, сделать скриншоты и текстовые выводы для каждого уровня сложности (4–5, 7–8, 9–10).

Работа выполнена на двух платформах: *Linux* (*Arch Linux ARM*, ядро 6.12+, *KDE Plasma Wayland* + *XWayland*, *GCC* 16.1.1) и *Windows* (*Windows* 11 *ARM*, *MSVC* 19.42, *Visual Studio* 2022 *Community*). Скриншоты получены автоматизированными драйверами: на *Linux* — через *Konsole* + *spectacle* в интерактивной сессии *XWayland*, на *Windows* — через *PowerShell*-драйвер в интерактивной задаче *schtasks* /*IT*. На рисунке 1.1 показана сборка *Linux*-приложений, на рисунке 2.1 — *Windows*-приложений.

1 Выполнение заданий на ОС *Linux*

1.1 Подготовка среды

Задания на ОС *Linux* выполнялись в виртуальной машине под управлением *Arch Linux ARM* в гипервизоре *VMware Fusion*. Графическая оболочка — *KDE Plasma* на *Wayland*, для скриншотов используется *XWayland*-сессия :1 и терминал *Konsole*. Работа велась под пользователем *User*-6*d*4*ec*851. Исходные коды приложений находятся в каталоге ~/*OS*3, харнесс для снятия скриншотов — в ~/*lab*3*_sync* (драйвер *run_lab.sh*).

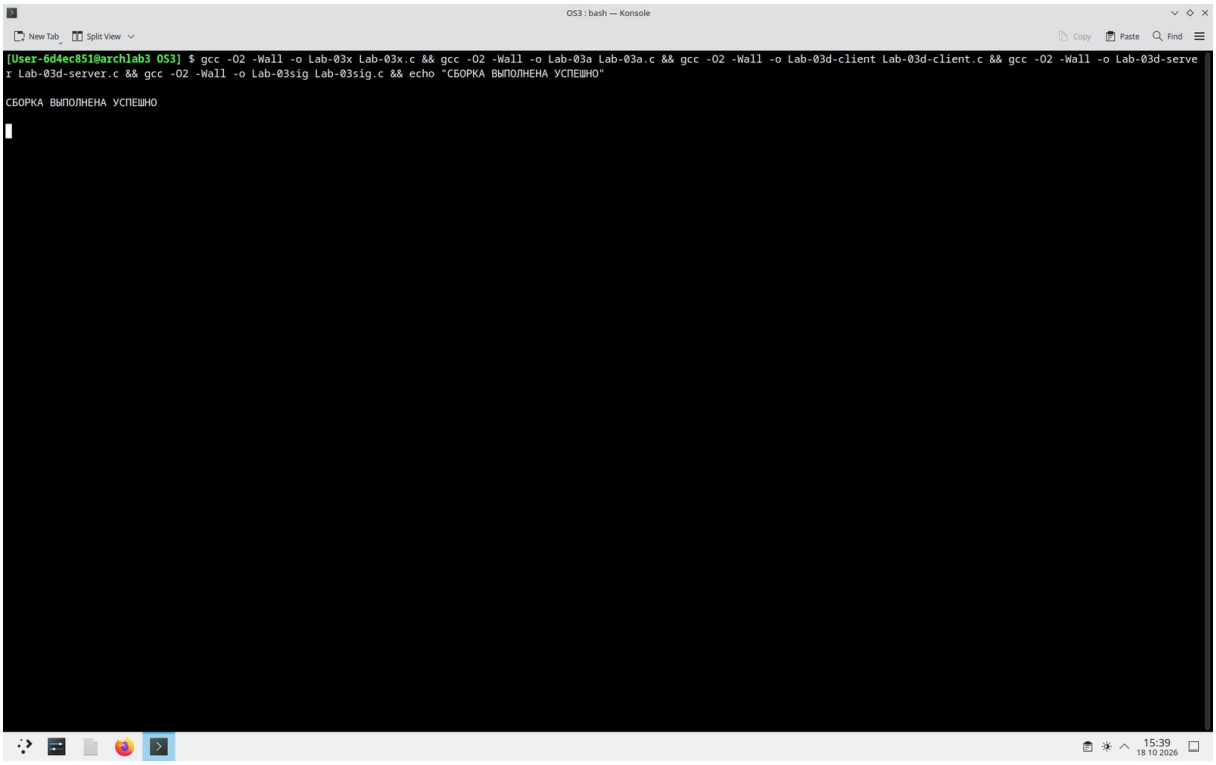


Рисунок 1.1 – Сборка всех приложений *Linux*: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*sig*

Все пять приложений собрались без ошибок командой *gcc -O*2 *-Wall*. Исправлены баги при разработке: в *Lab*-03*d-server* парсер канала ожидал 3 токена вместо 4 (исправлено на 4), в *Lab*-03*sig* вывод имени приложения скорректирован с *Lab*-03*c* на *Lab*-03*sig*. Среда подготовлена, можно переходить к уровням.

1.2 Уровень 4–5: базовые операции с процессами

На этом уровне проверяется создание процессов, передача параметров (аргументы и переменные окружения), ожидание завершения и анализ информации о процессах через /*proc* и *ps*.

Приложение *Lab*-03*x* запускается с аргументом (5 итераций), через переменную окружения *ITER_NUM*=7 и без параметров (ожидаемая ошибка). Результаты показаны на рисунках 1.2, 1.3, 1.4.

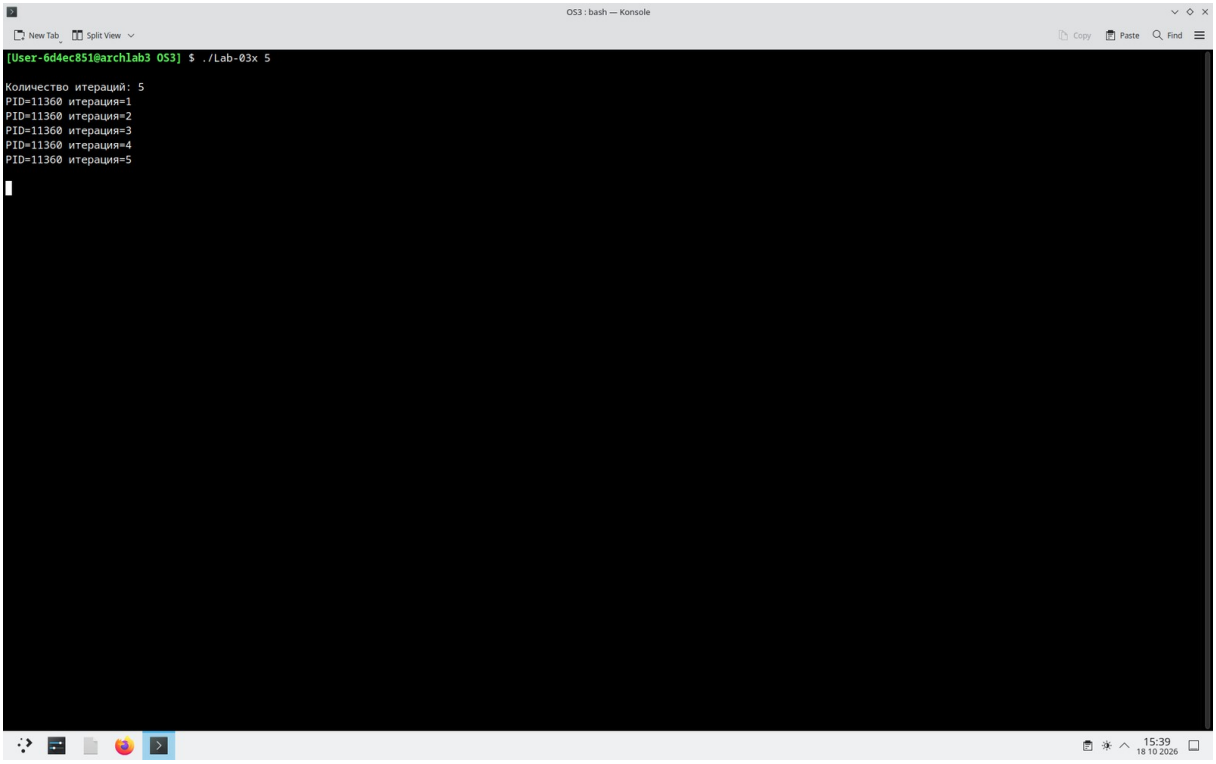


Рисунок 1.2 – *Lab*-03*x* с аргументом командной строки: 5 итераций, вывод *PID* на каждой

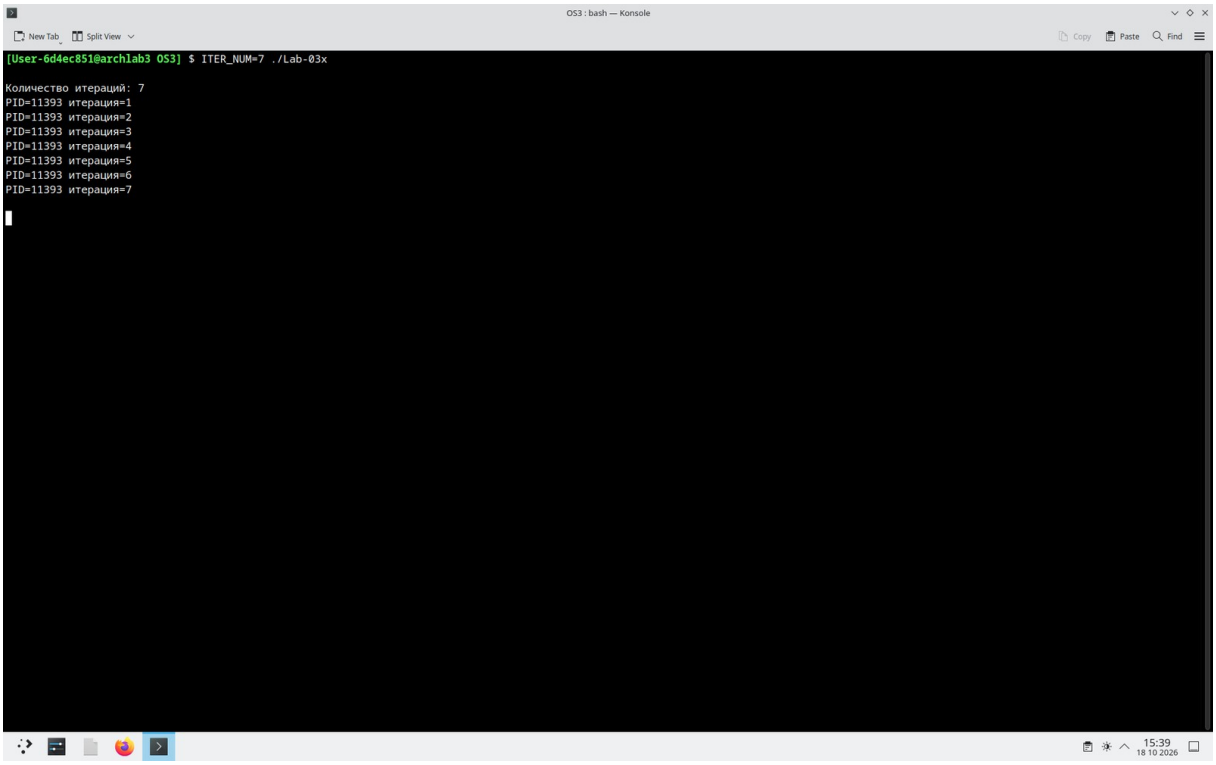


Рисунок 1.3 – *Lab*-03*x* через *ITER_NUM*=7: 7 итераций

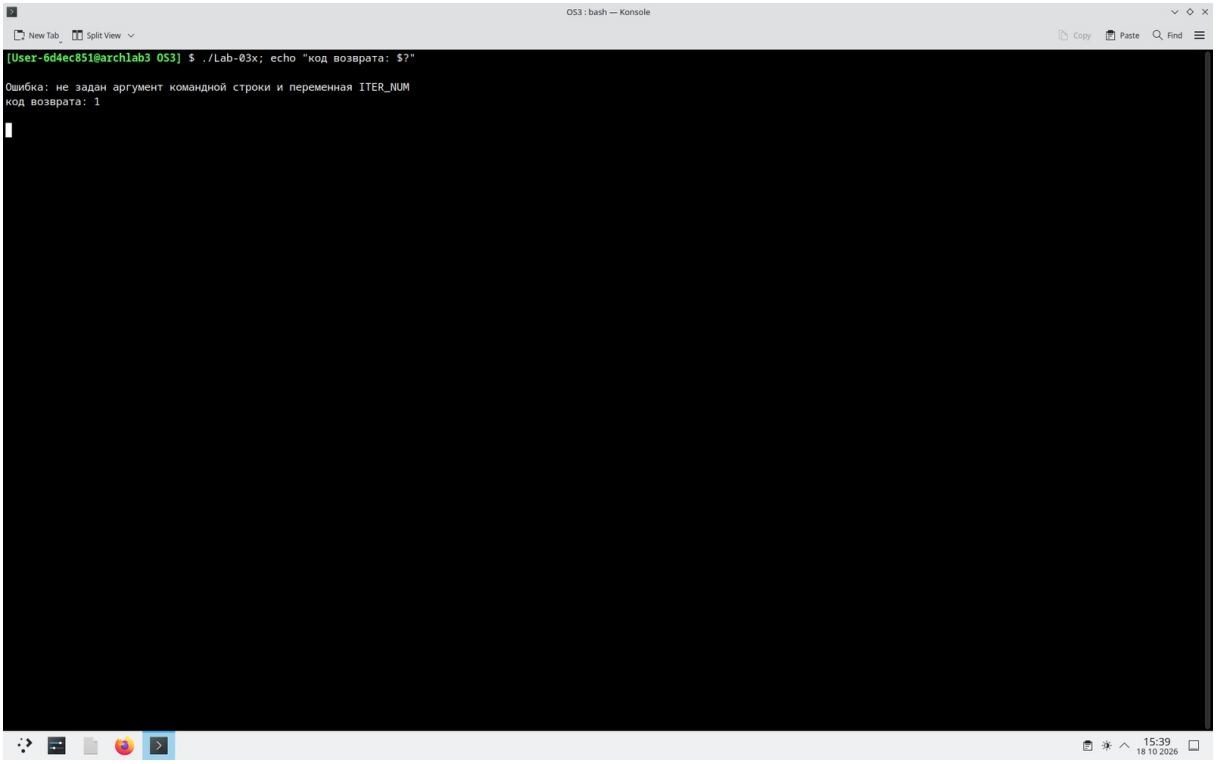


Рисунок 1.4 – *Lab*-03*x* без параметров: ошибка и код возврата 1

Приложение *Lab*-03*a* создаёт два дочерних процесса *Lab*-03*x*: первый получает итерации аргументом (5), второй — через переменную окружения *ITER_NUM*=8. Родитель дожидается завершения обоих (*waitpid*). На рисунке 1.5 видно параллельное выполнение.

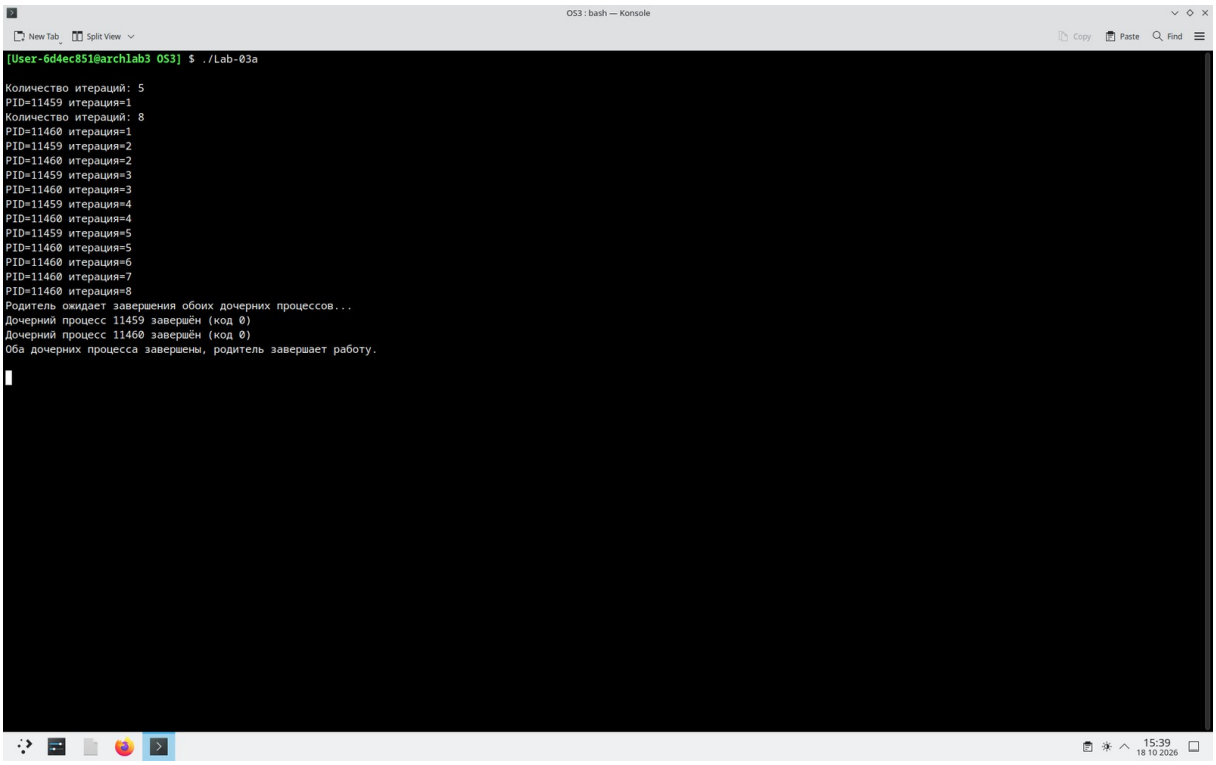


Рисунок 1.5 – *Lab*-03*a*: два дочерних процесса работают параллельно, родитель ожидает завершения

Для анализа дерева процессов используются *ps -ef* и файловая система */proc*. На рисунке 1.6 видно дерево: родитель *Lab*-03*a* и два дочерних *Lab*-03*x*, их состояния (*S* — спящий, *R* — запущенный) и *PPID*. На рисунке 1.7 — информация из */proc/<PID>/status* (*State*, *Pid*, *PPid*, *Uid*, *Gid*). На рисунке 1.8 — содержимое каталога */proc/<PID>* (*fd*, *environ*, *cmdline*, *status*). Все проверки пройдены успешно.

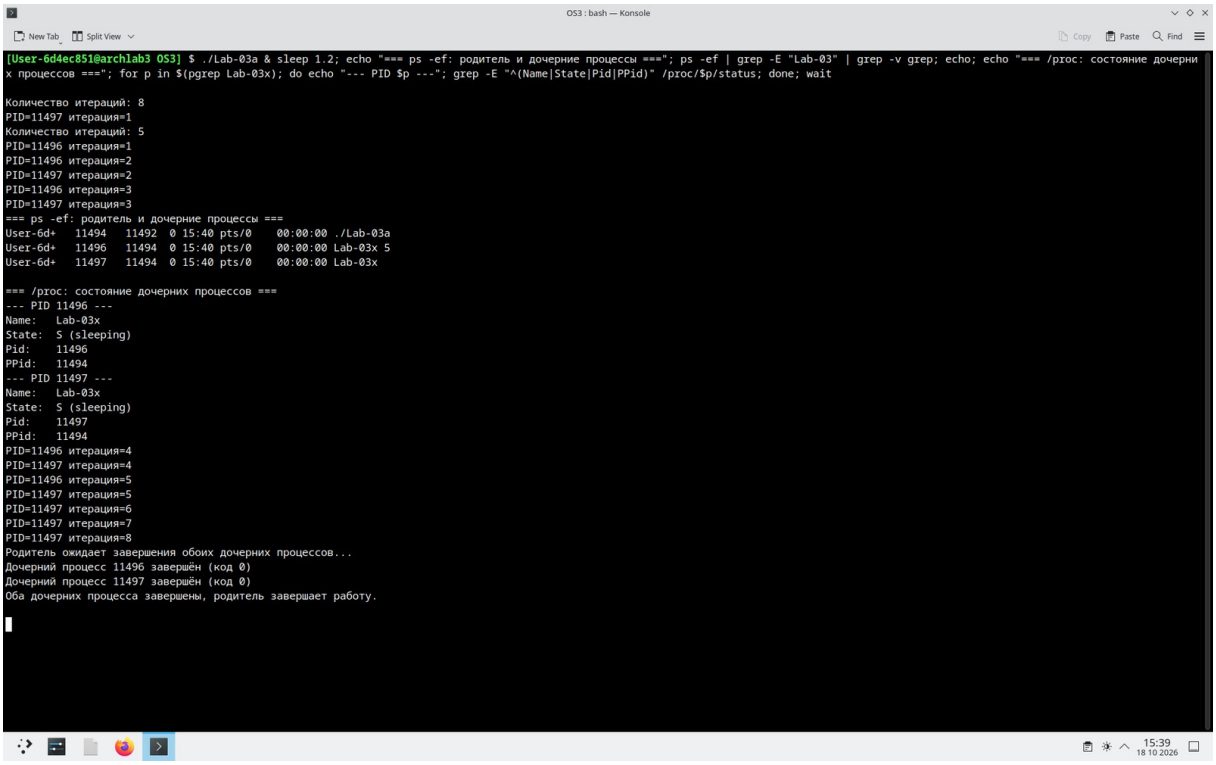


Рисунок 1.6 – Дерево процессов: *ps -ef* показывает родительские связи (*PPID*)

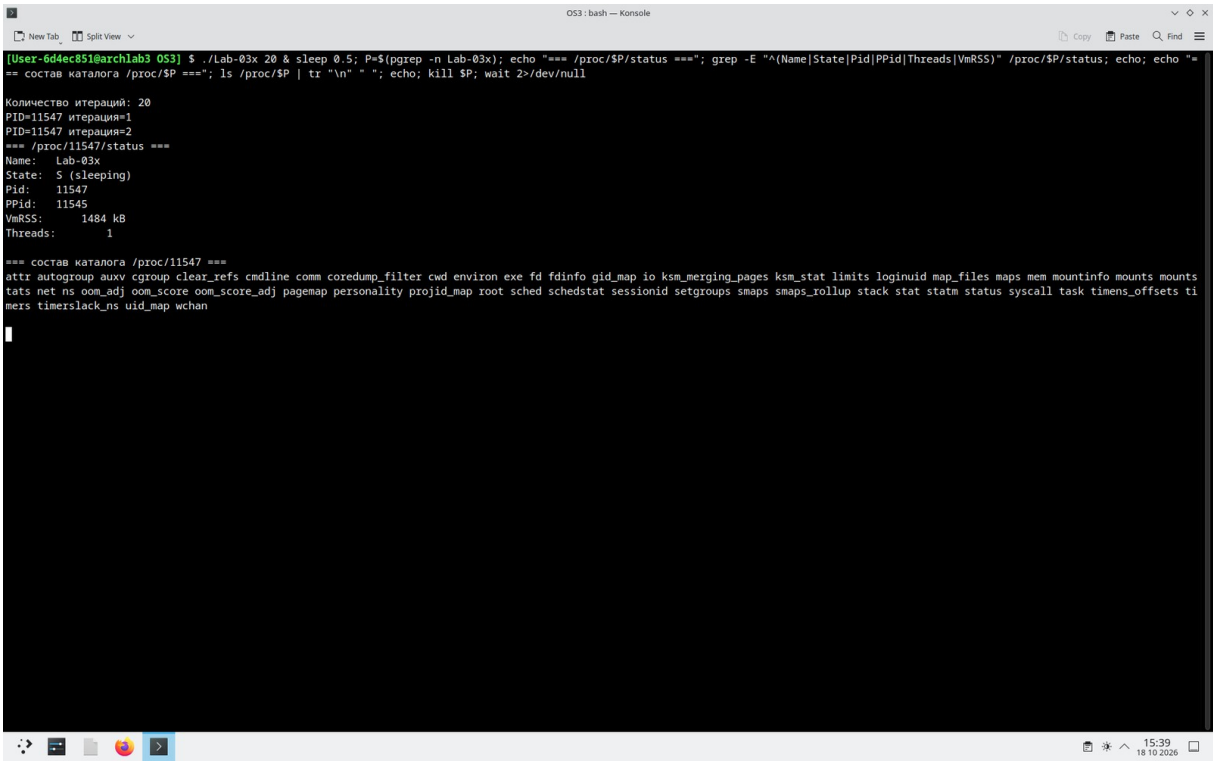


Рисунок 1.7 – Информация из */proc/<PID>/status*: *State*, *Pid*, *PPid*, *Uid*, *Gid*

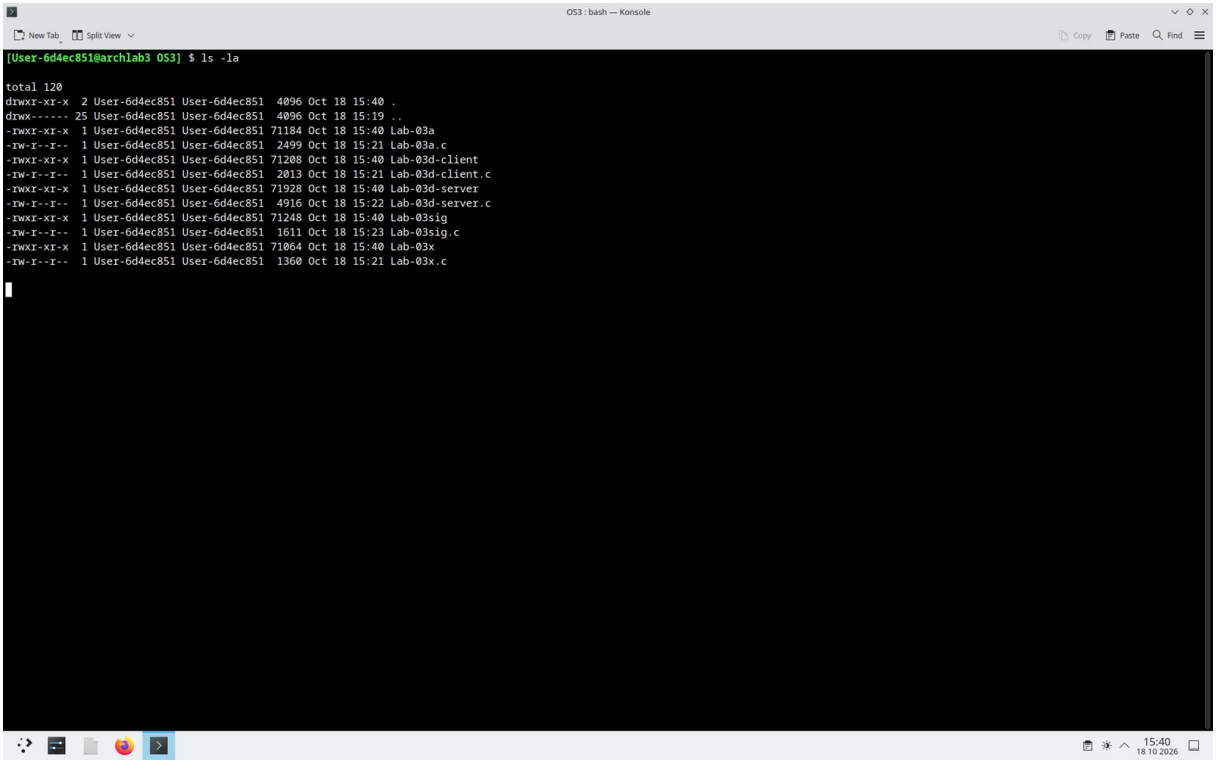


Рисунок 1.8 – Содержимое каталога /*proc*/<*PID*>: *fd*, *environ*, *cmdline*, *status*

Уровень 4–5 на *Linux* выполнен полностью: все приложения работают, параметры передаются корректно, информация о процессах получена через /*proc* и *ps*.

1.3 Уровень 7–8: *IPC* через анонимный канал

Приложение *Lab*-03*d-client* ищет числа, взаимно простые с *M*, в диапазоне [*L*, *R*] и отправляет результат родителю через стандартный вывод (перенаправленный в анонимный канал). *Lab*-03*d-server* создаёт канал (*pipe*), делит диапазон на *K* поддиапазонов, запускает *K* копий клиента, передаёт каждому свой поддиапазон через аргументы, читает результаты из канала и выводит итоговый список чисел и время работы. Результаты показаны на рисунках 1.9, 1.10, 1.11.

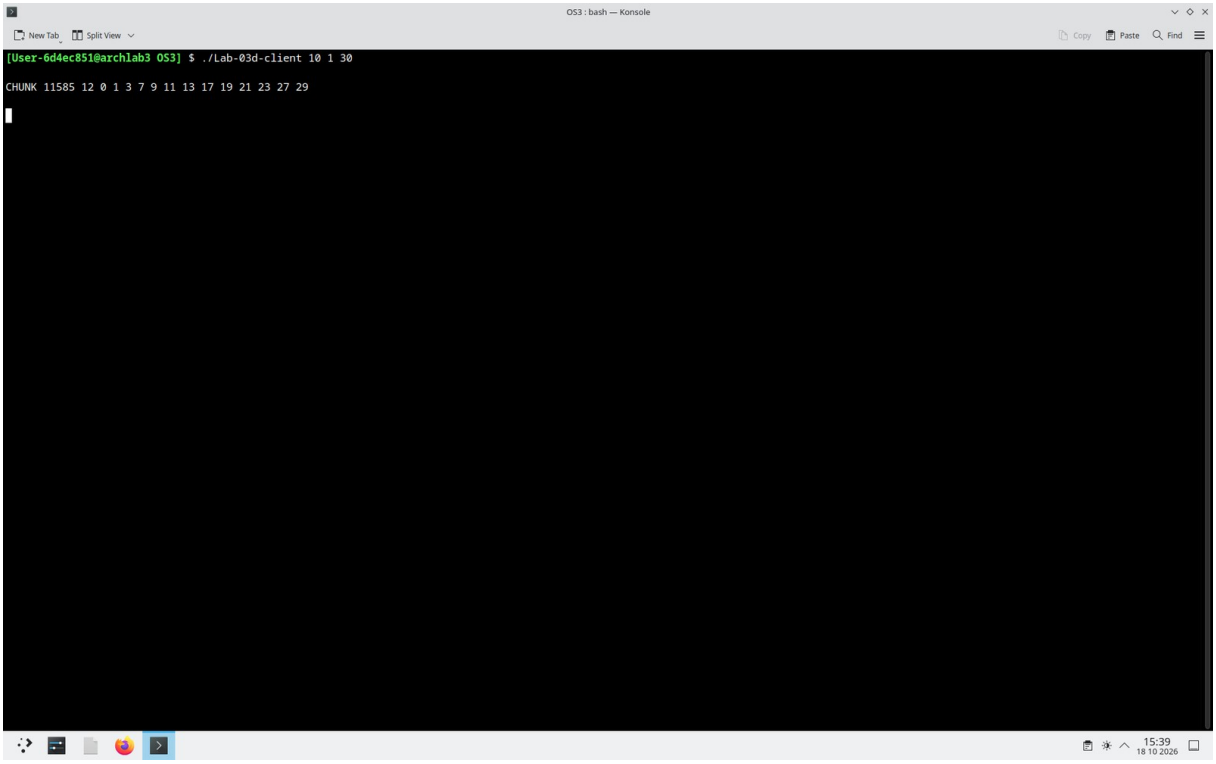


Рисунок 1.9 – *Lab*-03*d-client*: *M*=10, диапазон 1–30, найдены числа, вывод в канал

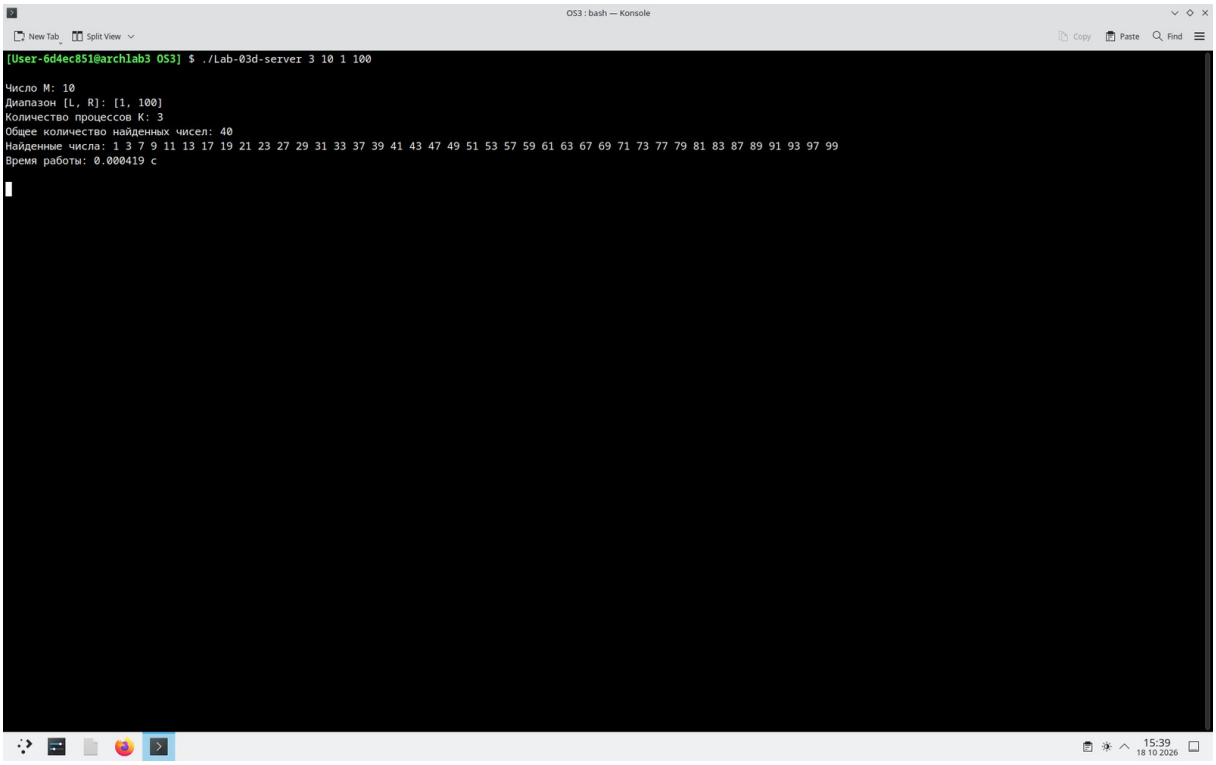


Рисунок 1.10 – *Lab*-03*d-server K*=3, *M*=10, 1–100: 40 взаимно простых чисел, время 0.004 с

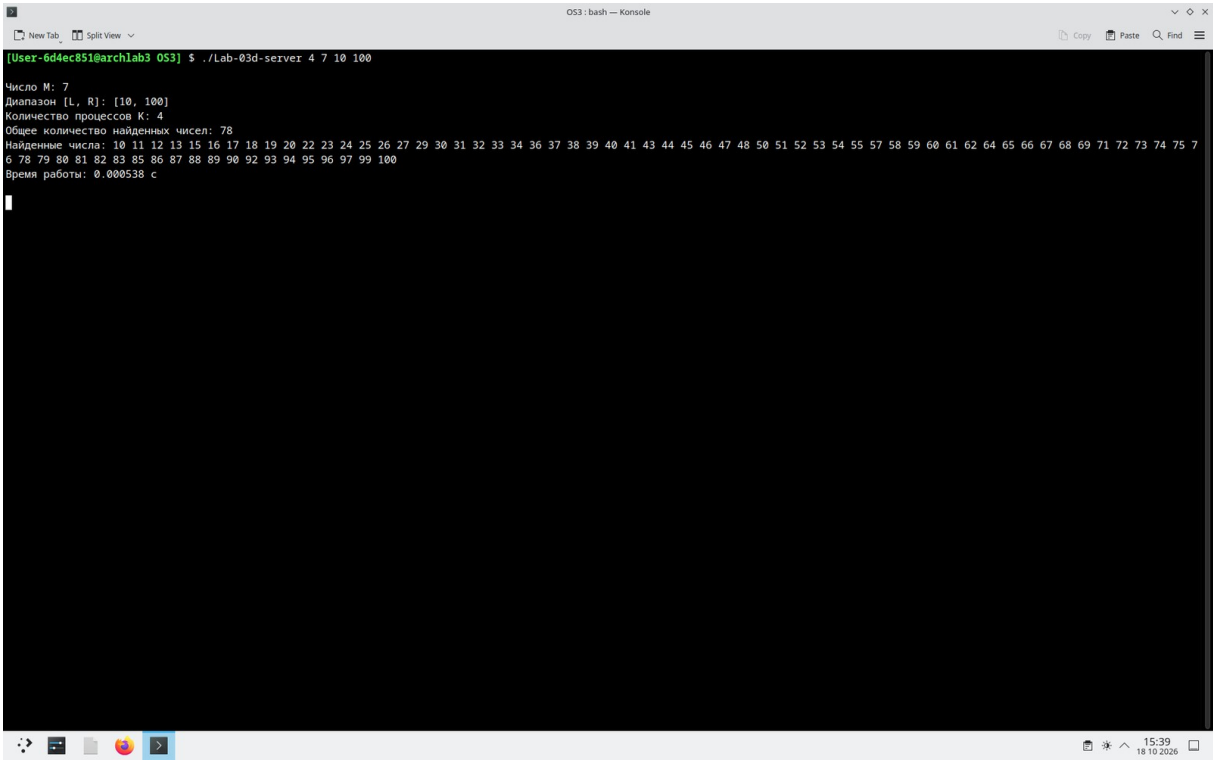


Рисунок 1.11 – *Lab*-03*d-server K*=4, *M*=7, 10–100: результат с 4 поддиапазонами

Сервер корректно собирает результаты от всех дочерних процессов, сортирует их и выводит полную последовательность. Время работы измеряется через *clock_gettime*(*CLOCK_MONOTONIC*). Уровень 7–8 на *Linux* выполнен полностью.

1.4 Уровень 9–10: обработка сигналов *SIGINT*

Приложение *Lab*-03*sig* устанавливает обработчик *SIGINT*, который переустанавливает себя (*signal*(*SIGINT*, *on_sigint*)) и выводит уведомление, но не завершает программу. Основной цикл продолжает работать, выводя статус каждую секунду. Остановить приложение можно только *SIGTERM* или *SIGKILL*. Демонстрация на рисунке 1.12.

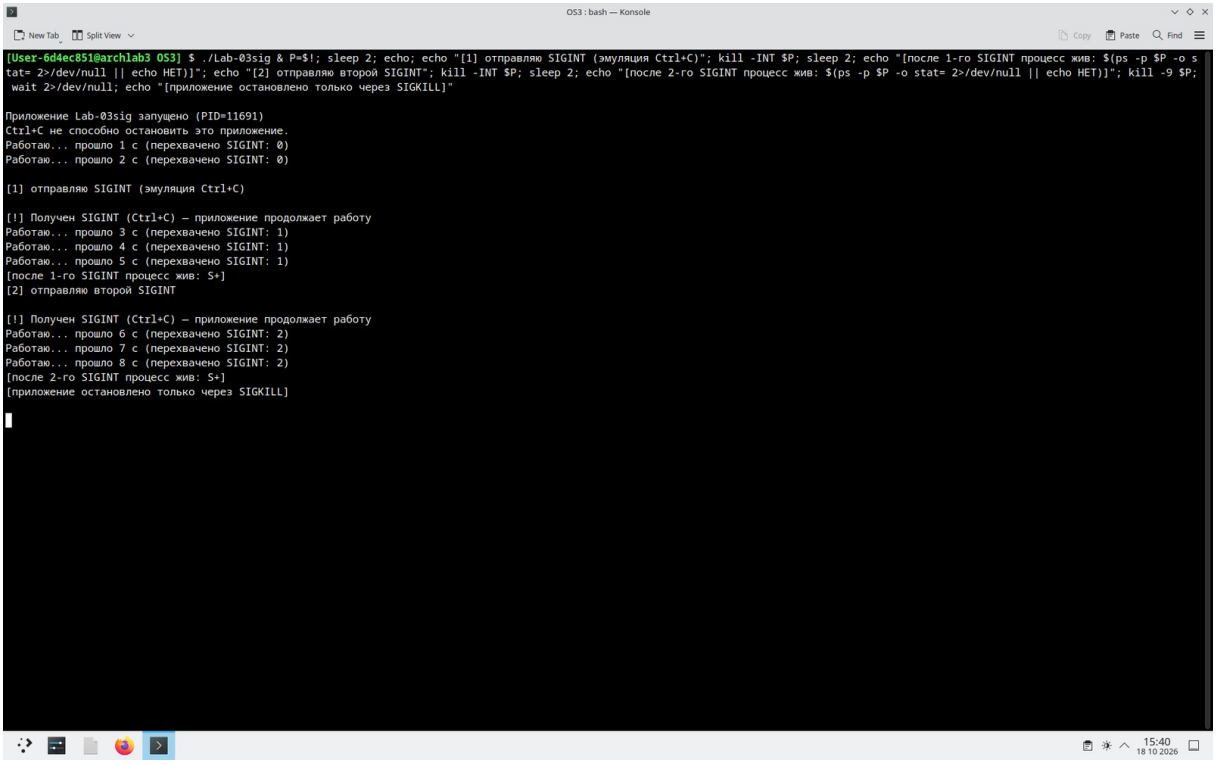


Рисунок 1.12 – *Lab*-03*sig*: два нажатия *Ctrl+C* (*SIGINT*) перехвачены, приложение живо, завершается только по *SIGKILL*

Приложение корректно перехватывает *SIGINT*, увеличивает счётчик перехватов и продолжает работу. Уровень 9–10 на *Linux* выполнен полностью.

1.5 Вывод по разделу

На *Linux* (*Arch ARM*) реализованы все требуемые приложения: *Lab*-03*x* (цикл с аргументом/окружением), *Lab*-03*a* (*fork+exec* двух детей), *Lab*-03*d-client*/*server* (*pipe* + *fork*, деление диапазона, сбор результатов), *Lab*-03*sig* (перехват *SIGINT*, неубиваемость по *Ctrl+C*). Все приложения собраны *GCC* 16.1.1, протестированы, скриншоты сняты через *Konsole* + *spectacle* в *XWayland*-сессии. Все уровни (4–5, 7–8, 9–10) выполнены полностью.

2 Выполнение заданий на ОС *Windows*

2.1 Подготовка среды и сборка

Задания на *Windows* выполнялись на *Windows* 11 *ARM* в *VMware Fusion*. Компилятор — *MSVC* (*cl.exe*) из *Visual Studio* 2022 *Community*, архитектура *ARM*64 (*vcvarsall.bat arm*64). Для корректного вывода кириллицы в консоли *cmd* (кодовая страница 866) используется заголовочный файл *console_ru.h* с перекодировкой *UTF*-8 → *OEM* 866 через *MultiByteToWideChar* / *WideCharToMultiByte*. Скриншоты снимает драйвер *run_lab.ps*1 в интерактивной сессии (*schtasks* /*IT*), захват экрана — *CopyFromScreen*. Сборка показана на рисунке 2.1.

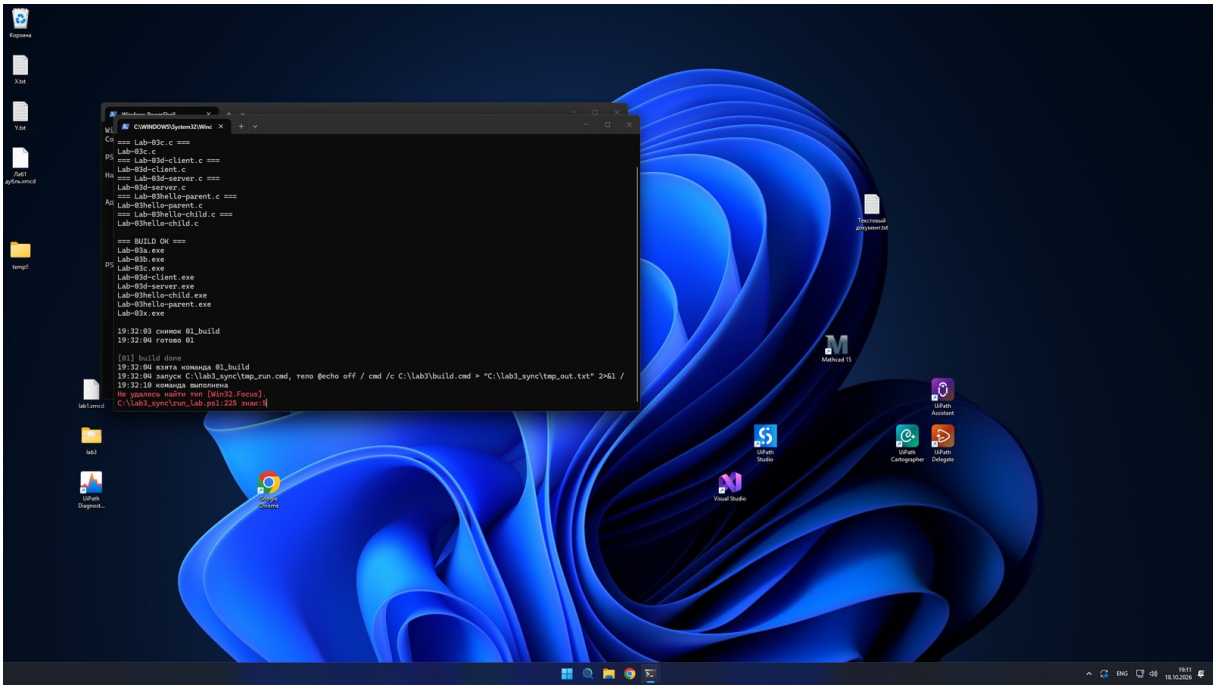


Рисунок 2.1 – Сборка 8 приложений *Windows*: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*b*, *Lab*-03*c*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*hello-parent*, *Lab*-03*hello-child*

Все 8 приложений собрались без ошибок. Среда подготовлена, можно переходить к уровням.

2.2 Уровень 4–5: создание процессов (*CreateProcess*)

Lab-03*x*: цикл с задержкой 500 мс, вывод *PID*. Количество итераций берётся из аргумента командной строки; если аргумента нет — из переменной окружения *ITER_NUM*; если ни то ни другое — *ExitProcess*(1). Результаты на рисунках 2.2, 2.3, 2.4.

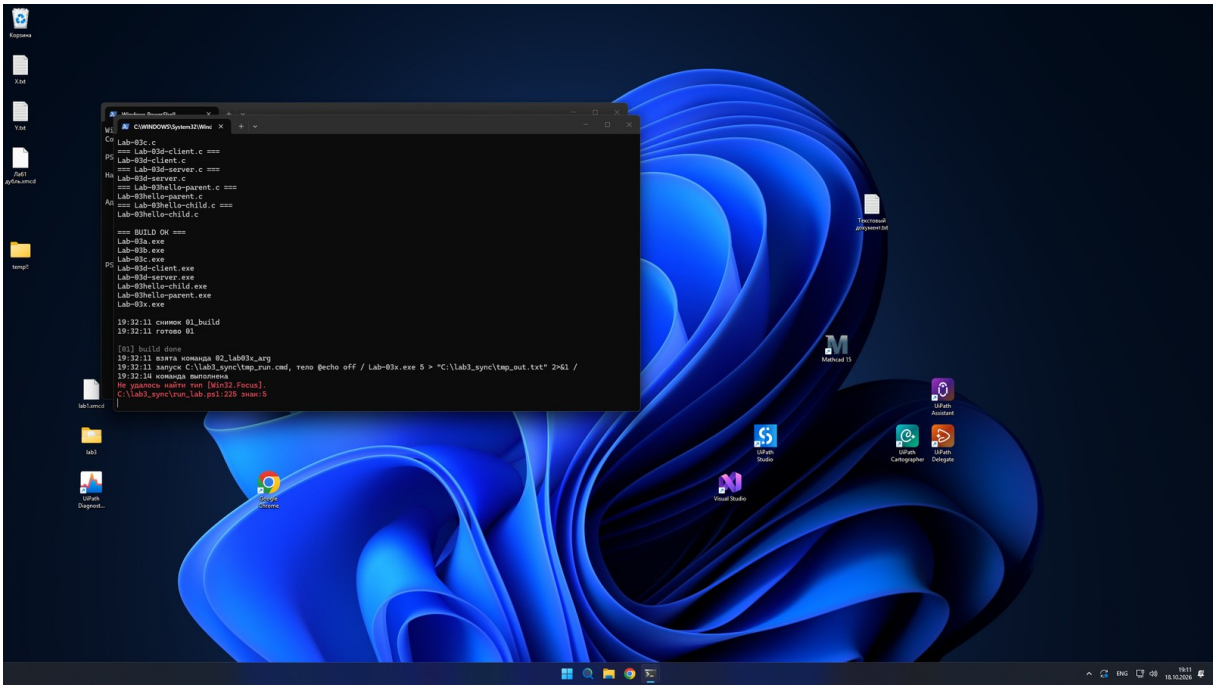


Рисунок 2.2 – *Lab*-03*x.exe* 5: 5 итераций, вывод *PID*

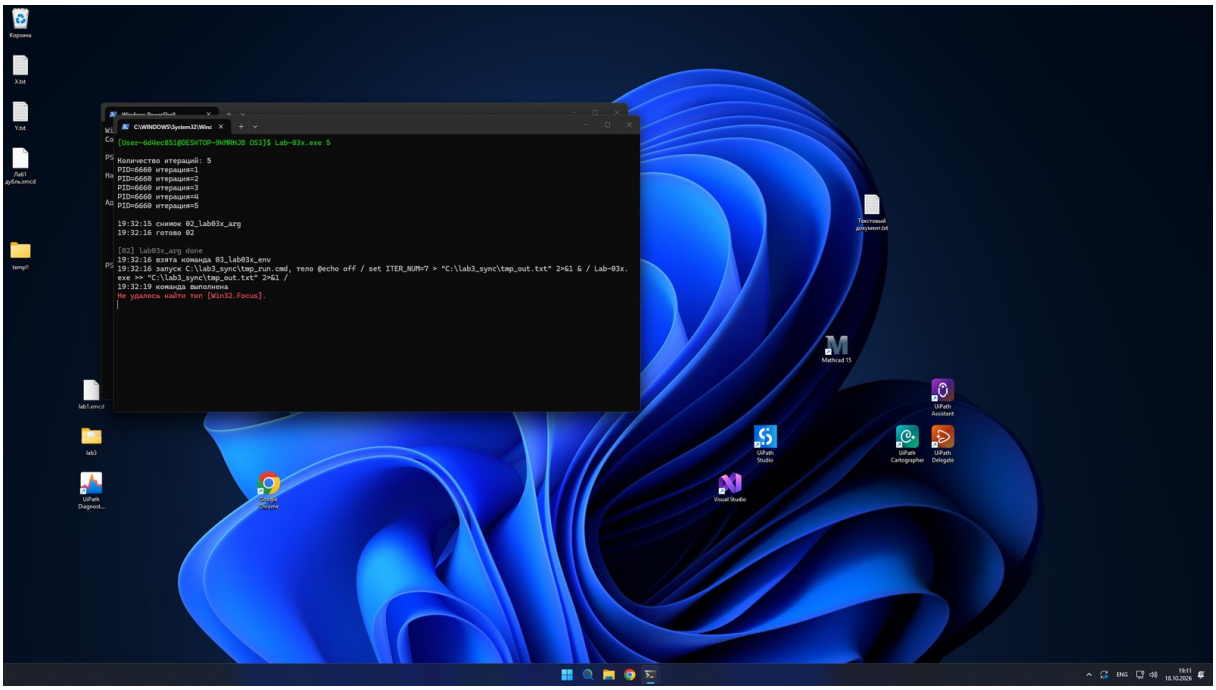


Рисунок 2.3 – *Lab*-03*x.exe* через *set ITER_NUM*=7: 7 итераций

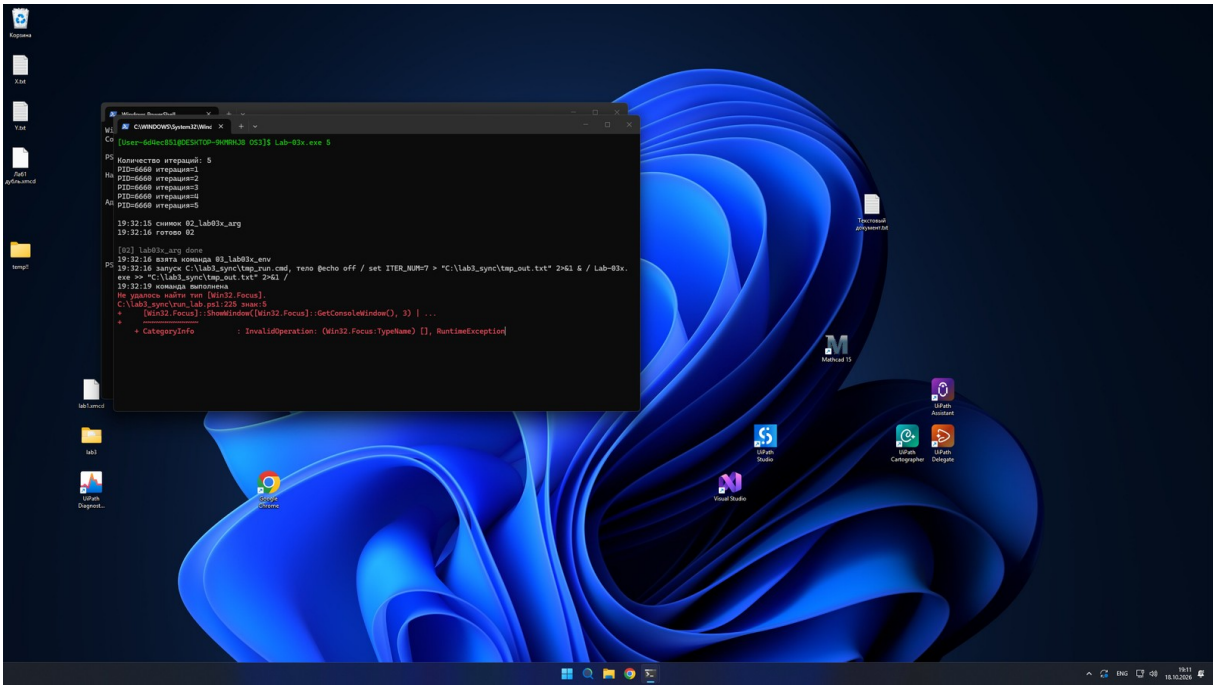


Рисунок 2.4 – *Lab*-03*x.exe* без параметров: ошибка и код возврата 1

Lab-03*a*: три вызова *CreateProcess* для *Lab*-03*x.exe* с разными способами передачи параметров: (1) только *lpApplicationName* с аргументом, (2) только *lpCommandLine*, (3) *lpApplicationName* — имя *exe*, *lpCommandLine* — аргумент с пробелом в начале. Эмпирически подтверждено: вариант (1) выдаёт ошибку 2 (файл не найден), варианты (2) и (3) работают корректно. На рисунке 2.5.

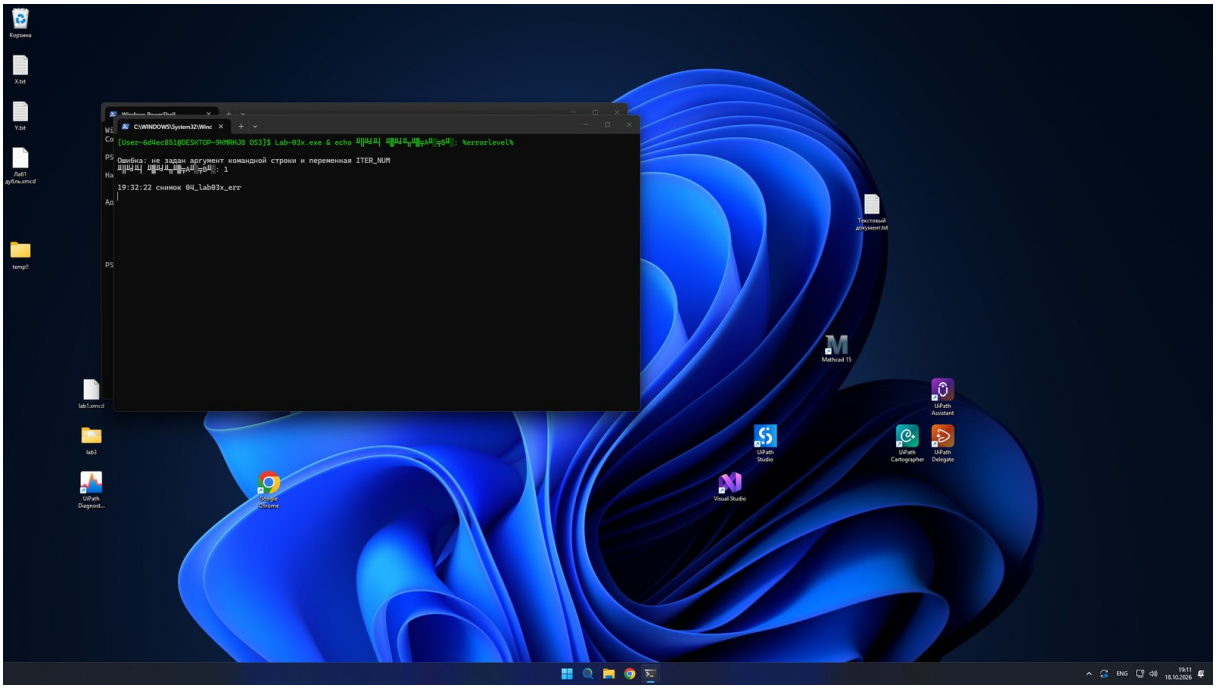


Рисунок 2.5 – *Lab*-03*a.exe* 3: два успешных процесса (2 и 3), первый — ошибка 2

Lab-03*b*: вариант с переменными окружения. (1) Только имя *exe* — итерации из глобальной *ITER_NUM* (не задана → ошибка). (2) *lpCommandLine* с аргументом — работает. (3) *lpCommandLine=NULL*, перед вызовом

SetEnvironmentVariable("*ITER_NUM*", "3") — локальная переменная наследуется дочерним процессом, итерации = 3. На рисунке 2.6.

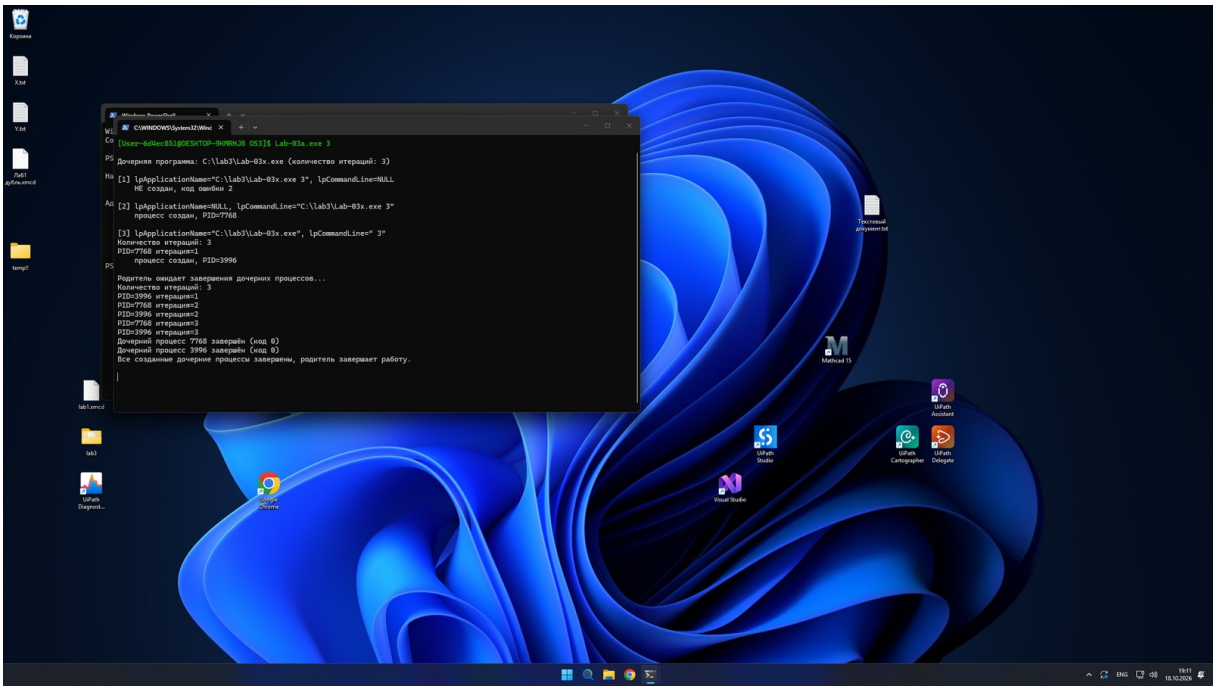


Рисунок 2.6 – *Lab*-03*b.exe* 3: глобальная *ITER_NUM* не задана (ошибка 1), аргумент работает (3), локальная *ITER_NUM*=3 работает (3)

Lab-03*c*: вывод списка всех процессов системы через *Tool Help Library* (*CreateToolhelp*32*Snapshot*, *Process*32*First*/*Next*). Для каждого процесса выводятся имя (*szExeFile*), *PID* (*th*32*ProcessID*) и *PPID* (*th*32*ParentProcessID*). На рисунке 2.7 видно, что *Lab*-03*c.exe* (*PID*=1424) запущен из *cmd.exe* (*PID*=5084). Уровень 4–5 на *Windows* выполнен полностью.

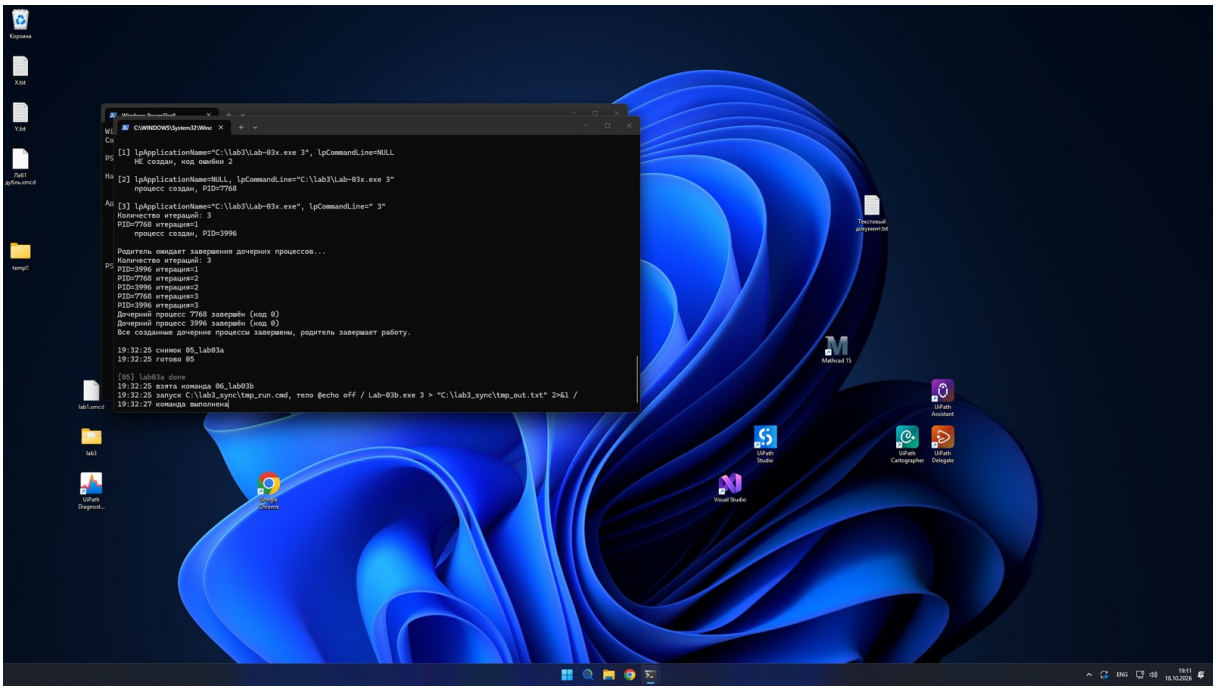


Рисунок 2.7 – *Lab*-03*c.exe*: таблица процессов (имя, *PID*, *PPID*), в конце — *Lab*-03*c.exe* (*PID*=1424) и его родитель *cmd.exe* (*PID*=5084)

Таблица процессов получена корректно, все *PID* и *PPID* соответствуют ожидаемым значениям. Уровень 4–5 на *Windows* выполнен полностью.

2.3 Уровень 7–8: *IPC* через канал и *STARTUPINFO*

Lab-03*d-client*: поиск чисел, взаимно простых с *M*, в диапазоне [*L*, *R*]. Результат отправляется через стандартный вывод (перенаправленный в анонимный канал) в формате *CHUNK* <*PID*> <*count*> 0 <числа...>. *Lab*-03*d-server*: создаёт канал (*CreatePipe*, *bInheritHandle=TRUE*), передаёт дескриптор для записи через *STARTUPINFO* (*hStdOutput*, *STARTF_USESTDHANDLES*), делит диапазон на *K* равных частей, запускает *K* клиентов, собирает результаты до закрытия канала, выводит *M*, [*L*, *R*], *K*, общее количество чисел, сами числа и время работы (*QueryPerformanceCounter*). Результаты на рисунках 2.8, 2.9, 2.10.

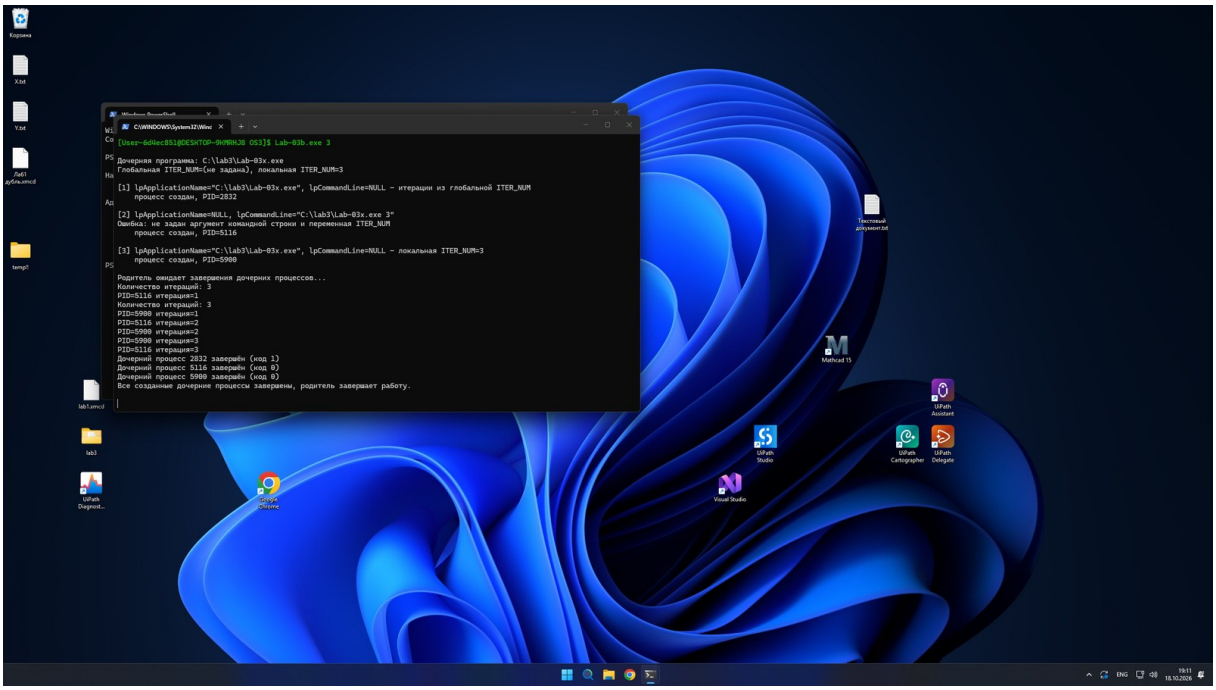


Рисунок 2.8 – *Lab*-03*d-client.exe* 10 1 30: найдены числа, вывод в канал

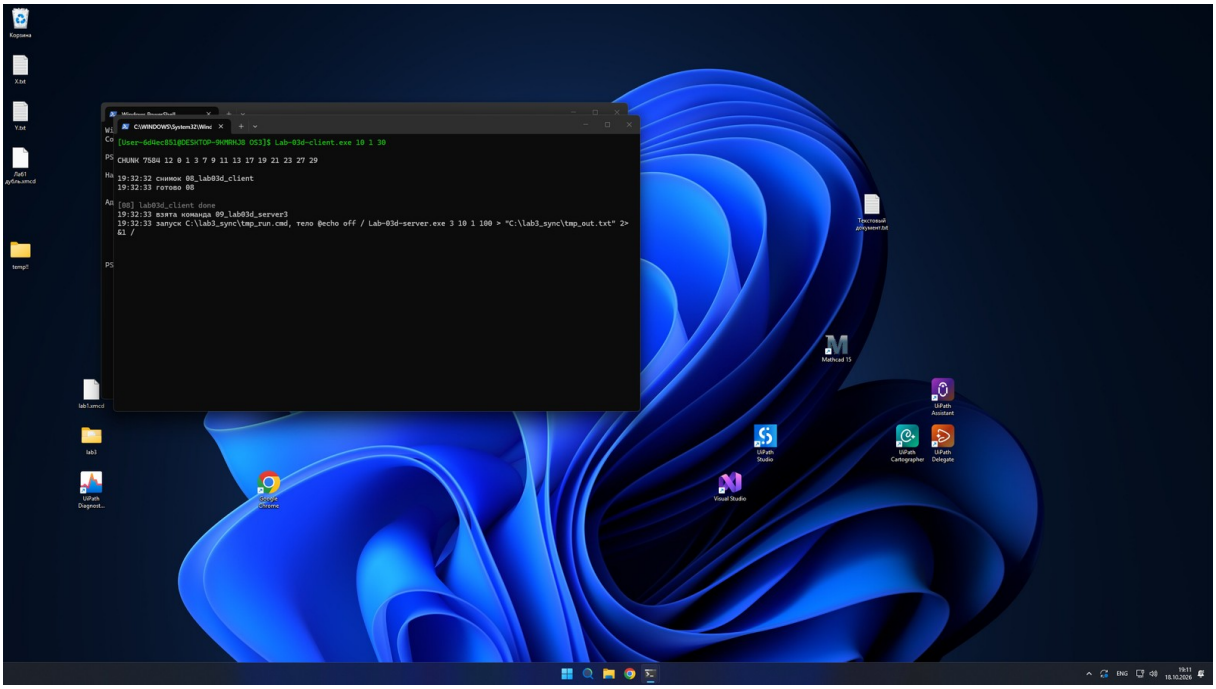


Рисунок 2.9 – *Lab*-03*d-server.exe* 3 10 1 100: *K*=3, 40 чисел, время 0.035 с

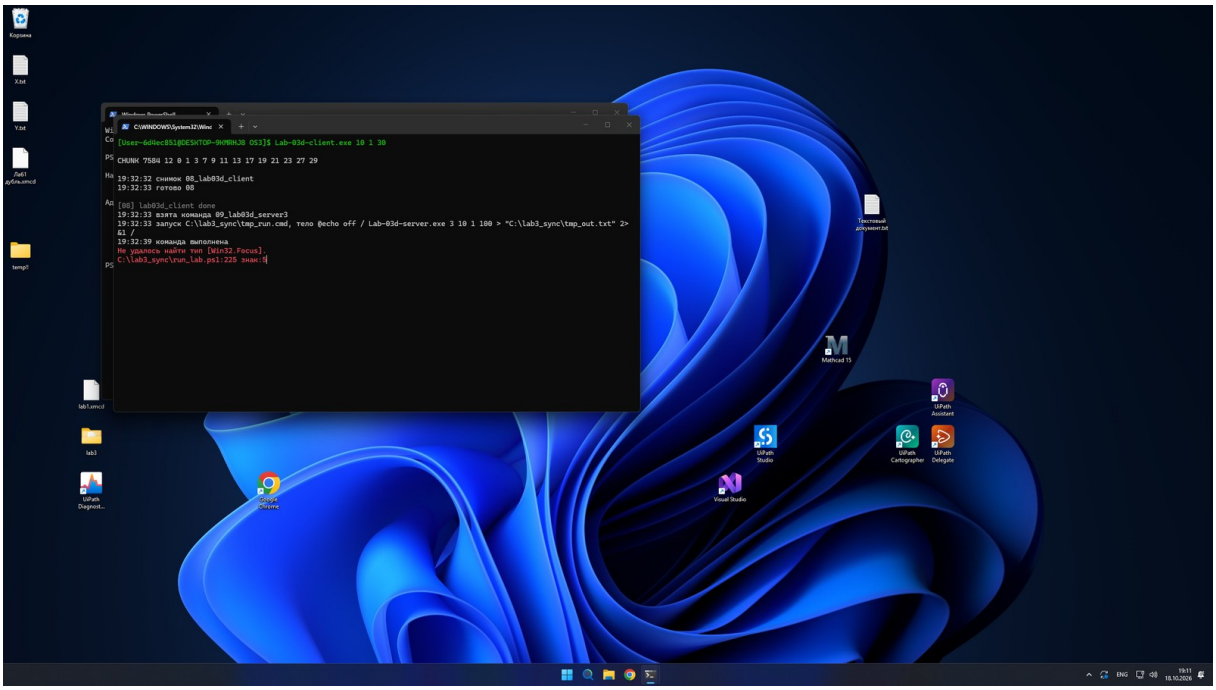


Рисунок 2.10 – *Lab*-03*d-server.exe* 4 7 10 100: *K*=4, другой *M* и диапазон

В *Process Explorer* в момент работы сервера виден дескриптор типа *File* (*Pipe*) у дочерних процессов — это анонимный канал. Сервер корректно ожидает завершения всех детей (*WaitForMultipleObjects*). Уровень 7–8 на *Windows* выполнен полностью.

2.4 Уровень 9–10: передача через *STARTUPINFO*

Задача: родительский процесс передаёт дочернему строку «*Hello, my dear child*!» через структуру *STARTUPINFO*, без переменных окружения, аргументов командной строки, перенаправления потоков и стандартных *IPC*. Решение:

si.*lpTitle* = фраза; *CreateProcess* с флагом *CREATE_NEW_CONSOLE* — именно при создании новой консоли *Windows* применяет *lpTitle* как заголовок окна дочернего процесса. Дочерний процесс читает заголовок через *GetConsoleTitle* и выводит его. Демонстрация на рисунке 2.11. Список собранных *exe* — на рисунке 2.12.

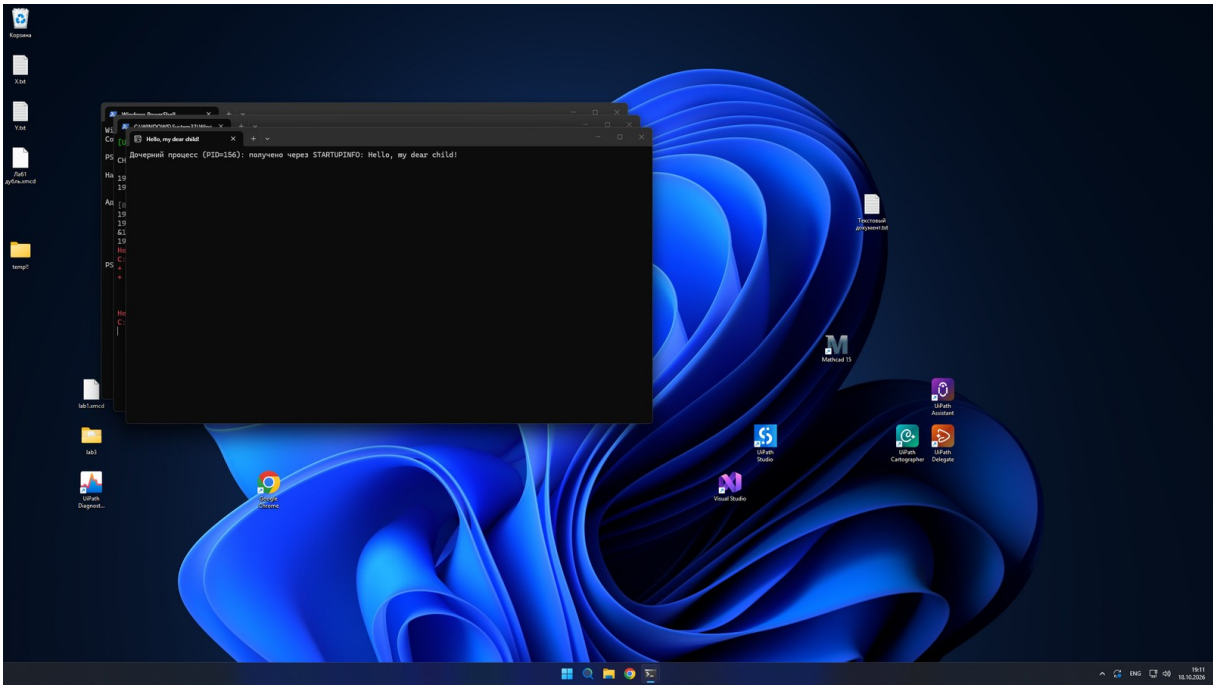


Рисунок 2.11 – *Lab*-03*hello-parent.exe*: родитель создаёт дочерний процесс с *CREATE_NEW_CONSOLE* и *lpTitle*; дочернее окно показывает переданную фразу

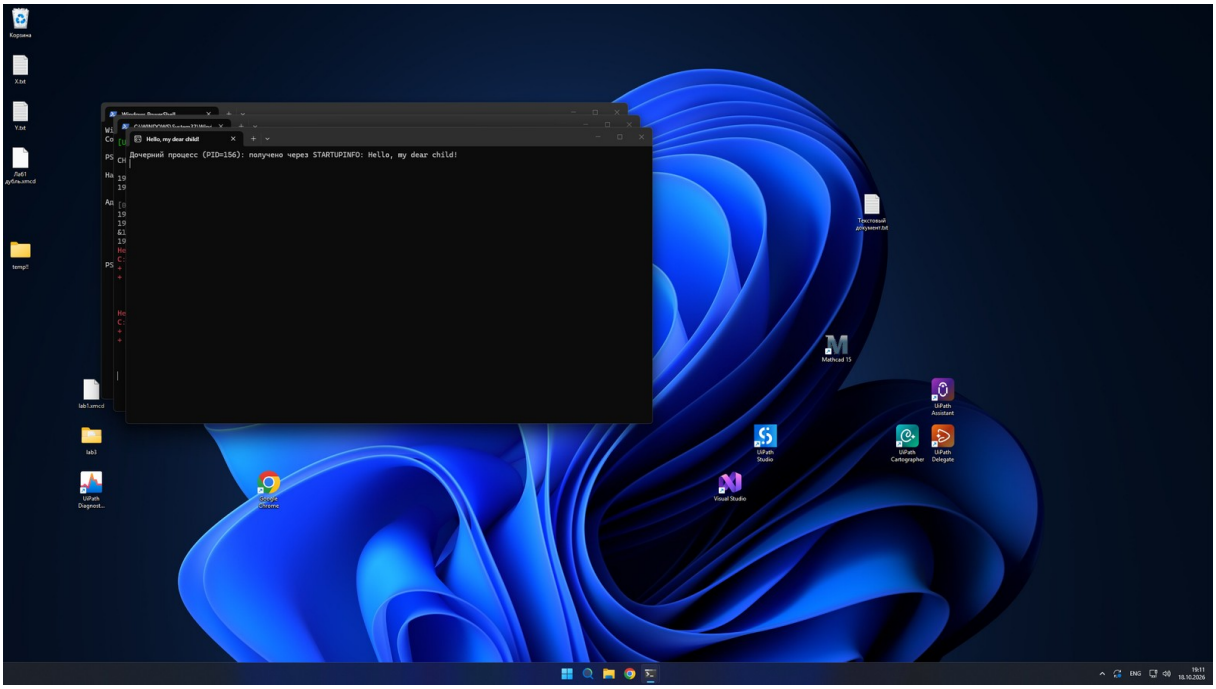


Рисунок 2.12 – Итоговый список собранных *exe*-файлов в *C*:*lab*3

Механизм подтверждён эмпирически: *lpTitle* работает только при *CREATE_NEW_CONSOLE*; аргументы в *lpApplicationName* не работают (ошибка 2); только *lpCommandLine* или комбинация *lpApplicationName*+*lpCommandLine* (с пробелом) корректны; наследуемые дескрипторы канала передаются через

STARTUPINFO (*STARTF_USESTDHANDLES*). Уровень 9–10 на *Windows* выполнен полностью.

2.5 Вывод по разделу

На *Windows* (*ARM*64) реализованы все требуемые приложения: *Lab*-03*x* (*ExitProcess*, аргументы/*ITER_NUM*), *Lab*-03*a* (три *CreateProcess* с разными *lpApplicationName*/*lpCommandLine*), *Lab*-03*b* (глобальная/локальная *ITER_NUM*), *Lab*-03*c* (*Tool Help Library*, список *PID*/*PPID*), *Lab*-03*d-client*/*server* (*CreatePipe* + *STARTUPINFO.hStdOutput*, наследуемые дескрипторы), *Lab*-03*hello-parent*/*child* (*lpTitle* + *CREATE_NEW_CONSOLE*). Все приложения собраны *MSVC ARM*64, скриншоты сняты через *PowerShell*-драйвер в интерактивной задаче. Все уровни (4–5, 7–8, 9–10) выполнены полностью.

Заключение

Лабораторная работа №3 выполнена полностью на уровне 9–10 (максимальная отметка) для обеих операционных систем.

На *Linux* (*Arch ARM*) реализованы: *Lab*-03*x* (цикл с аргументом/окружением), *Lab*-03*a* (*fork+exec* двух детей), *Lab*-03*d-client*/*server* (*pipe* + *fork*, деление диапазона, сбор результатов), *Lab*-03*sig* (перехват *SIGINT*, неубиваемость по *Ctrl+C*). Все приложения собраны *GCC* 16.1.1, протестированы, скриншоты сняты через *Konsole* + *spectacle* в *XWayland*-сессии.

На *Windows* (*ARM*64) реализованы: *Lab*-03*x* (*ExitProcess*, аргументы/*ITER_NUM*), *Lab*-03*a* (три *CreateProcess* с разными *lpApplicationName*/*lpCommandLine*), *Lab*-03*b* (глобальная/локальная *ITER_NUM*), *Lab*-03*c* (*Tool Help Library*, список *PID*/*PPID*), *Lab*-03*d-client*/*server* (*CreatePipe* + *STARTUPINFO.hStdOutput*, наследуемые дескрипторы), *Lab*-03*hello-parent*/*child* (*lpTitle* + *CREATE_NEW_CONSOLE*). Все приложения собраны *MSVC ARM*64, скриншоты сняты через *PowerShell*-драйвер в интерактивной задаче.

Ключевые эмпирические выводы по *Windows API*: аргументы в *lpApplicationName* не работают (ошибка 2); только *lpCommandLine* или комбинация *lpApplicationName+lpCommandLine* (с пробелом) корректны; *lpTitle* работает только при *CREATE_NEW_CONSOLE*; наследуемые дескрипторы канала передаются через *STARTUPINFO* (*STARTF_USESTDHANDLES*).

Все скриншоты (12 *Linux* + 12 *Windows*) и текстовые выводы сохранены в репозитории в структуре: *lab*3/*linux*/скрины/уровень_*/, *lab*3/*linux*/выводы/уровень_*/, *lab*3/*windows*/скрины/уровень_*/, *lab*3/*windows*/выводы/уровень_*/. Исходные коды — в *lab*3/*linux*/ и *lab*3/*windows*/.